

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №5
по дисциплине «Построение и анализ алгоритмов»
Тема: Поиск набора подстрок в строке.

Студент гр. 4343

Гизатов Я.Р.

Преподаватель

Жангиров Т. Р.

Санкт-Петербург

2026

Задание 1

Разработайте программу, решающую задачу точного поиска набора образцов.

Вход:

Первая строка содержит текст $(T, 1 \leq |T| \leq 1000000T)$.
Вторая - число n ($1 \leq n \leq 3000$), каждая следующая из n строк содержит шаблон из набора $P = \{p_1, \dots, p_n\}$ $1 \leq |p_i| \leq 75$. Все строки содержат символы из алфавита $\{A, C, G, T, N\}$.

Выход:

Все вхождения образцов из P в T . Каждое вхождение образца в текст представить в виде двух чисел - i p . Где i - позиция в тексте (нумерация начинается с 1), с которой начинается вхождение образца с номером p (нумерация образцов начинается с 1). Строки выхода должны быть отсортированы по возрастанию, сначала номера позиции, затем номера шаблона.

Входные данные

NTAG

3

TAGT

TAG

T

Выходные данные

2 2

2 3

Задание 2

Используя реализацию точного множественного поиска, решите задачу точного поиска для одного образца с джокером. В шаблоне встречается специальный символ, именуемый джокером (wild card), который "совпадает" с любым символом. По заданному содержащему шаблоны образцу P необходимо найти все вхождения P в текст T . Например,

образец $ab???c?ab???c?$ с джокером $??$ встречается дважды в тексте $habvssbababcsaxhabvssbababcsax$.

Символ джокер не входит в алфавит, символы которого используются в T . Каждый джокер соответствует одному символу, а не подстроке неопределённой длины. В шаблон входит хотя бы один символ не джокер, т.е. шаблоны вида $???$ недопустимы. Все строки содержат символы из алфавита $\{A, C, G, T, N\}$

Вход:

Текст ($T, 1 \leq |T| \leq 100000$)

Шаблон ($P, 1 \leq |P| \leq 40$)

Символ джокера

Выход:

Строки с номерами позиций вхождений шаблона (каждая строка содержит только один номер). Номера должны выводиться в порядке возрастания.

Задание 3

Вариант 3. Вычислить длину самой длинной цепочки из суффиксных ссылок и самой длинной цепочки из конечных ссылок в автомате.

Задание 1

Выполнение работы

Для выполнения задания была разработана программа, реализующая алгоритм Ахо-Корасик. Сначала из всех заданных образцов строится бор (префиксное дерево), каждый узел которого хранит переходы по символам алфавита $\{A, C, G, T, N\}$. Затем для узлов бора вычисляются fail-ссылки, позволяющие продолжать поиск при несовпадении, а также дополнительные выходные ссылки для быстрого обнаружения всех образцов, оканчивающихся в текущей позиции. После построения автомата выполняется проход по тексту: состояние автомата обновляется для каждого символа, и при нахождении одного или нескольких образцов сохраняются их номера и стартовые позиции. В конце все найденные вхождения

сортируются по возрастанию позиции и номера образца и выводятся в требуемом формате.

Основные структуры данных

`struct Node`

Структура, представляющая узел бора (автомата). Массив `next[5]` хранит индексы дочерних узлов для каждого из пяти символов алфавита {A, C, G, T, N} (значение -1 означает отсутствие перехода). Поле `fail` — индекс узла, в который происходит переход по `fail`-ссылке (суффиксная ссылка). Поле `out` — индекс узла, на который ведёт выходная ссылка (для быстрого перечисления всех образцов, оканчивающихся в данном узле или его суффиксах). Вектор `pat` содержит номера образцов, заканчивающихся в данном узле. Конструктор инициализирует `next` значениями -1, `fail` и `out` нулём или -1.

`vector<Node> nodes`

Динамический массив, хранящий все узлы автомата. Индекс 0 соответствует корню. При добавлении нового узла (например, при вставке образца или при создании переходов) структура `Node` добавляется в конец вектора.

`vector<vector<int>> ans`

Массив, индексируемый позициями в тексте (от 1 до длины текста). Для каждой позиции `i` хранит вектор номеров образцов, которые начинаются в этой позиции. Используется для сбора всех найденных вхождений перед финальной сортировкой и выводом.

`queue<int> q`

Очередь целых чисел, используемая при построении `fail`-ссылок в алгоритме Ахо-Корасик. В неё помещаются узлы бора (начиная с прямых потомков корня), затем в ширину обрабатываются их переходы, вычисляются `fail`-ссылки и переназначаются недостающие переходы.

Описание алгоритма

Алгоритм, реализованный в программе, представляет собой классический алгоритм Ахо-Корасик для множественного точного поиска подстрок. Сначала строится бор (префиксное дерево) из всех заданных образцов: для каждого образца последовательно добавляются его символы в дерево, при необходимости создаются новые узлы, а в терминальных узлах сохраняются номера соответствующих образцов. Затем выполняется построение fail-ссылок (суффиксных ссылок) с помощью обхода в ширину: для каждого узла вычисляется узел, в который нужно перейти при несовпадении по текущему символу, а также формируются выходные ссылки, ведущие к ближайшему узлу, содержащему хотя бы один образец. Также на этом этапе для каждого узла доопределяются недостающие переходы (если переход по символу отсутствует, он направляется в состояние, полученное по fail-ссылке). После построения автомата выполняется проход по тексту: текущее состояние обновляется для каждого символа согласно построенным переходам; в текущем узле и по всем его выходным ссылкам извлекаются номера образцов, вычисляется их начальная позиция в тексте, и эти вхождения сохраняются. В конце все накопленные вхождения сортируются по возрастанию позиции и номера образца и выводятся в указанном формате.

Оценка сложности

Построение бора требует времени, пропорционального суммарной длине всех образцов L , так как каждый символ каждого образца добавляется в дерево за константное время. Затем построение fail-ссылок и доопределение недостающих переходов выполняется с помощью обхода в ширину по всем узлам: для каждого из N узлов обрабатываются все символы алфавита, что даёт $O(N \cdot |A|)$, где $|A|$ – длина алфавита операций. Проход по тексту длины $|T|$ занимает $O(|T|)$ шагов, плюс обход по выходным ссылкам для выдачи всех найденных образцов — в сумме $O(|T| + M)$, где M — общее количество вхождений. После сбора результатов требуется их сортировка, которая в худшем случае добавляет

$O(M \log n)$ операций. Таким образом, итоговая временная сложность составляет

$$O(L + N \cdot |A| + |T| + M \log n).$$

Оценка по памяти

По памяти основную нагрузку создаёт хранение узлов бора: массив из N структур, каждая из которых содержит 5 целочисленных переходов, fail-ссылку, out-ссылку и вектор номеров образцов. Это требует $O(N \cdot |A|)$ памяти для переходов плюс $O(N)$ для служебных полей и суммарно $O(n)$ для хранения всех номеров образцов в векторах. Дополнительно выделяется массив ответов `ans` размером $|T| + 1$, элементы которого являются векторами; в них сохраняются все M найденных вхождений, что даёт $O(|T| + M)$ памяти. Также используется очередь для BFS, вмещающая до N узлов. Следовательно, общая память оценивается как

$$O(N \cdot |A| + |T| + M).$$

Задание 2

Выполнение работы

Для выполнения задания была разработана программа, реализующая поиск шаблона с символами-джокерами на основе алгоритма Ахо-Корасик. Сначала шаблон разбивается на непрерывные сегменты, не содержащие джокеров; каждый сегмент рассматривается как отдельный образец, для которого запоминается его смещение относительно начала шаблона. Все полученные сегменты вставляются в бор, после чего для каждого узла вычисляются суффиксные и выходные ссылки (с ленивой рекурсивной инициализацией). Затем выполняется проход по тексту: состояние автомата обновляется по каждому символу, и при обнаружении окончания какого-либо сегмента вычисляется потенциальная начальная позиция всего шаблона в тексте, для которой увеличивается специальный счётчик. В конце проверяются все возможные позиции, и если для данной позиции счётчик совпадает с общим количеством сегментов, эта позиция выводится как место вхождения шаблона.

Основные структуры данных

struct Node

Структура, представляющая узел бора (автомата) в реализации с ленивыми вычислениями переходов. Массив `son[ALPHABET_SIZE]` хранит указатели на дочерние узлы по каждому из пяти символов {A, C, G, T, N}. Массив `go[ALPHABET_SIZE]` — кэшированные переходы автомата (включая переходы по fail-ссылкам). Указатель `parent` — на родительский узел. `suffLink` — суффиксная ссылка (fail-ссылка). `up` — выходная ссылка (сжатая суффиксная ссылка на ближайший терминальный узел). Поле `charToParent` — символ, по которому узел был получен из родителя. `isLeaf` — флаг, является ли узел терминальным (окончанием хотя бы одного сегмента). Вектор `leafPatternNumber` хранит номера сегментов (индексы в списке сегментов), которые заканчиваются в данном узле.

Node* root;

Глобальный указатель на корневой узел бора. Корень не имеет родителя, его суффиксная и выходная ссылки указывают на него самого. Через корень начинается вставка сегментов и все обходы текста.

vector<int> pat_len, pat_offset

Два параллельных вектора, хранящих информацию о сегментах шаблона, полученных после разбиения по джокеру. `pat_offset` — смещение сегмента в исходном шаблоне (индекс первого символа сегмента). `pat_len` — длина сегмента. Размер каждого вектора равен количеству сегментов `k`. Используются для вычисления позиции начала всего шаблона при обнаружении сегмента и для проверки границ текста.

vector<int> cnt

Массив счётчиков, индексируемый потенциальными начальными позициями шаблона в тексте (от 0 до $|T|-|P|$). Каждый раз, когда при обработке текста обнаруживается окончание какого-либо сегмента, вычисляется соответствующая начальная позиция кандидата `start_candidate`, и счётчик `cnt[start_candidate]` увеличивается на 1. После прохода по тексту

позиции, у которых счётчик достиг значения k (общее число сегментов), считаются успешными вхождениями шаблона.

Описание алгоритма

Программа решает задачу поиска всех вхождений шаблона, содержащего символы-джокеры (обозначающие любой символ алфавита), в заданный текст. Алгоритм работы следующий. Сначала шаблон разбивается на максимальные непрерывные сегменты, не содержащие джокеров. Для каждого сегмента запоминаются его смещение относительно начала шаблона и длина. Все полученные сегменты вставляются в бор (префиксное дерево), причём каждый терминальный узел хранит номер соответствующего сегмента. Затем для узлов бора рекурсивно вычисляются суффиксные ссылки (fail-ссылки) и выходные ссылки (up), позволяющие быстро переходить между узлами при несовпадении и находить все сегменты, оканчивающиеся в текущей позиции. После построения автомата выполняется проход по тексту слева направо. Текущее состояние автомата обновляется по каждому символу текста, и для этого состояния обходом по выходным ссылкам определяются все сегменты, которые завершились в данной позиции. Для каждого найденного сегмента вычисляется потенциальная начальная позиция всего шаблона в тексте (с учётом смещения сегмента и его длины), и соответствующий счётчик увеличивается на единицу. После обработки всего текста проверяются все возможные стартовые позиции (от 0 до длины текста минус длина шаблона). Если для данной позиции счётчик равен общему количеству сегментов, это означает, что все части шаблона совпали, и позиция выводится как результат (в 1-индексации).

Описание вспомогательных функций

`int char_to_idx(char c)`

Функция преобразует символ алфавита {A, C, G, T, N} в соответствующий индекс от 0 до 4. Для символа 'A' возвращает 0, 'C' — 1, 'G' — 2, 'T' — 3, 'N' — 4. Любой другой символ по умолчанию отображается

в 0 (буква 'A'). Используется для индексации массивов переходов в узле бора.

char idx_to_char(int i)

Выполняет обратное преобразование: по индексу от 0 до 4 возвращает соответствующий символ 'A', 'C', 'G', 'T', 'N'. При некорректном индексе возвращает '?'. Применяется в отладочном выводе (при флаге --logflag) для наглядного представления переходов по символам.

Node* getSuffLink(Node* v)

Вычисляет и возвращает суффиксную ссылку (fail-ссылку) для узла *v*. Реализация ленивая: если ссылка уже вычислена, она возвращается; иначе она вычисляется рекурсивно. Правило: для корня и его прямых потомков ссылка ведёт в корень. Для остальных узлов суффиксная ссылка получается как переход по символу `charToParent` из суффиксной ссылки родителя. Результат сохраняется в поле `suffLink` узла.

Node* getLink(Node* v, char c)

Возвращает состояние, в которое автомат переходит из узла *v* по символу *c* (с учётом суффиксных ссылок). Использует кэширование в массиве `go`. Если переход уже сохранён, он возвращается; иначе, если у узла есть сын по символу *c*, то переход ведёт в этого сына. Для корня при отсутствии сына переход ведёт обратно в корень. В противном случае переход вычисляется как `getLink(getSuffLink(v), c)`. Результат сохраняется в `v->go[idx]` для последующего использования.

Node* getUp(Node* v)

Возвращает выходную ссылку (сжатую суффиксную ссылку) для узла *v*. Эта ссылка ведёт к ближайшему узлу, достижимому по суффиксным ссылкам, который является терминальным (содержит хотя бы один сегмент). Вычисляется лениво: если узел уже имеет `up`, он возвращается; иначе берётся суффиксная ссылка узла. Если эта суффиксная ссылка ведёт в терминальный узел, то `up` указывает на неё. Если суффиксная ссылка — корень, то `up` тоже корень. Иначе рекурсивно вызывается `getUp` от

суффиксной ссылки. Используется для быстрого перечисления всех сегментов, оканчивающихся в текущей позиции текста.

void addString(const string& s, int patternNumber)

Вставляет строку *s* (сегмент шаблона) в бор. Начиная с корня, для каждого символа строки вычисляется его индекс, и если соответствующий сын отсутствует, создаётся новый узел. У нового узла устанавливаются поля *parent* и *charToParent*. После прохода всей строки текущий узел помечается терминальным (*isLeaf* = true), а номер сегмента *patternNumber* добавляется в его вектор *leafPatternNumber*. Эта функция вызывается для каждого сегмента шаблона после разбиения по джокерам.

Оценка сложности

Алгоритм состоит из нескольких этапов. Разбиение шаблона на сегменты и вставка их в бор выполняются за время $O(m)$, где m — длина шаблона, а L — суммарная длина сегментов ($L \leq m$). Построение суффиксных и выходных ссылок с ленивой мемоизацией требует $O(N \cdot |A|)$ операций, где N — количество узлов бора ($N \leq L + 1 \leq m + 1$), $|A| = 5$, т.е. $O(m)$. Основной этап — проход по тексту длины n . Для каждого символа выполняется константный переход по автомату, после чего происходит обход по выходным ссылкам (цепочка *up*). Каждый шаг этого обхода соответствует обнаружению окончания какого-либо сегмента, поэтому общее количество итераций цикла *while* по всем позициям текста равно M_seg — общему числу найденных вхождений сегментов. В худшем случае каждый сегмент может заканчиваться на каждой позиции текста, тогда $M_seg = n \cdot k$, где k — количество сегментов ($k \leq m$). Таким образом, временная сложность прохода по тексту составляет $O(n + n \cdot k) = O(n \cdot k)$. Финальная проверка позиций занимает $O(n)$. Итоговая временная сложность: $O(m + n \cdot k)$. При неблагоприятных условиях (короткие сегменты, часто повторяющиеся) она может достигать $O(n \cdot m)$.

Оценка памяти

Память алгоритма складывается из нескольких составляющих. Хранение текста T требует $O(|T|)$ памяти, хранение шаблона P — $O(|P|)$. Векторы `pat_len` и `pat_offset`, содержащие информацию о сегментах, занимают $O(k)$, где k — количество сегментов, причём $k \leq |P|$. Массив счётчиков `snt` имеет размер $|T| + 5$, что даёт $O(|T|)$. Основной объём памяти приходится на бор: количество узлов N не превышает суммарной длины всех сегментов плюс единица, то есть $N \leq |P| + 1$. Каждый узел содержит два массива переходов `son` и `go` размером $|A| = 5$ каждый, три служебных указателя `parent`, `suffLink`, `up`, символ `charToParent`, булев флаг `isLeaf`, а также вектор `leafPatternNumber`, в котором в сумме по всем узлам хранится k номеров сегментов. Таким образом, память под узлы составляет $O(N \cdot |A| + k)$. Учитывая, что $N = O(|P|)$ и $k = O(|P|)$, общая оценка памяти принимает вид $O(|T| + |P| \cdot |A|)$. При фиксированном размере алфавита $|A| = 5$ это упрощается до $O(|T| + |P|)$.

Задание 3

Выполнение работы

Данная программа строит бор по набору образцов, вычисляет для каждой вершины суффиксные и выходные ссылки по алгоритму Ахо-Корасик, а затем определяет максимальную длину цепочки суффиксных ссылок и максимальную длину цепочки выходных ссылок среди всех вершин автомата.

Основные структуры данных

Node

Структура, описывающая вершину бора (автомата). Хранит массив `next[26]` — переходы по буквам латинского алфавита (индексы дочерних вершин или -1, если перехода нет); целое число `fail` — индекс вершины, в которую ведёт суффиксная ссылка; целое число `out` — индекс вершины, в которую ведёт выходная ссылка (по ней переходят к терминальным вершинам); булево поле `term` — флаг, является ли вершина терминальной

(соответствует концу хотя бы одного образца). Нужна для построения и работы автомата Ахо-Корасик.

`vector<Node> trie`

Динамический массив, хранящий все вершины бора. Индекс 0 – корневая вершина. В процессе добавления образцов в бор создаются новые вершины, для каждой из которых инициализируется своя структура Node. Используется для представления всего автомата, включая переходы, суффиксные и выходные ссылки.

`queue<int> q`

Очередь целых чисел (индексов вершин), применяемая при обходе в ширину (BFS) для построения суффиксных ссылок. На начальном этапе в неё помещаются все непосредственные дети корня, затем последовательно обрабатываются вершины, их дети, что позволяет корректно вычислить fail-ссылки для всех узлов бора.

`vector<int> memo_fail, memo_out`

Векторы для мемоизации результатов рекурсивных вычислений. `memo_fail[v]` хранит длину цепочки суффиксных ссылок от вершины `v` до корня (глубину по `fail`), `memo_out[v]` – длину цепочки выходных ссылок. Изначально заполнены -1. Нужны для ускорения подсчёта максимальных длин, чтобы избежать повторных рекурсивных вызовов.

Описание алгоритма

Построение бора (Trie)

Программа считывает количество образцов и сами строки. Для каждого образца, начиная от корневой вершины (индекс 0), происходит движение по символам: если нужный переход по букве отсутствует, создаётся новая вершина, а в текущей вершине запоминается индекс дочерней. После добавления всех символов образца последняя вершина помечается как терминальная (флаг `term = true`). В итоге получается бор, содержащий все образцы, причём отсутствующие у корня переходы сразу доопределяются в корень (для упрощения дальнейших вычислений).

Построение суффиксных ссылок (fail)

Используется обход в ширину (очередь). Для всех непосредственных детей корня их fail-ссылка устанавливается в корень, а сами дети помещаются в очередь. Далее для каждой извлекаемой вершины v просматриваются все буквы: если у v есть дочерняя вершина u по букве c , то fail-ссылка для u устанавливается как результат перехода по той же букве c из fail-ссылки вершины v (то есть $\text{trie}[\text{trie}[v].\text{fail}].\text{next}[c]$). Если же перехода нет, то чтобы не обрабатывать отсутствие перехода в дальнейшем, он заменяется на переход из fail-ссылки текущей вершины – так формируются «сжатые» переходы. После обработки всех вершин каждый узел получает корректную суффиксную ссылку, ведущую в вершину, соответствующую наибольшему суффиксу пути от корня до текущей вершины, который также является префиксом какого-либо образца.

Построение выходных ссылок (out)

Для каждой вершины (кроме корня) определяется выходная ссылка: если fail-ссылка вершины ведёт в терминальный узел, то out этой вершины указывает на неё; в противном случае out копируется из out fail-ссылки. Такая ссылка позволяет быстро переходить к ближайшей терминальной вершине (концу образца) по цепочке суффиксных ссылок, что упрощает поиск всех вхождений.

Вычисление максимальных длин цепочек

Для всех вершин с помощью рекурсивных функций с мемоизацией (векторы `memo_fail` и `memo_out`) вычисляются: глубина цепочки суффиксных ссылок (количество переходов от вершины до корня по fail) и глубина цепочки выходных ссылок (количество переходов до ближайшей терминальной вершины по out). Затем находятся максимальные значения среди всех вершин – `max_fail` и `max_out`. Эти два числа выводятся на экран, а также при включённом флаге логирования записываются в файл `log3.txt` с подробными отладочными сообщениями о каждом шаге.

Оценка сложности

Алгоритм работает за $O(S * |A|)$, где S — суммарная длина всех образцов, $|A| = 26$ (размер алфавита). Поскольку $|A|$ фиксирован, это эквивалентно $O(S)$.

- Построение бора: каждый символ каждого образца обрабатывается один раз $O(S)$.
- Построение суффиксных ссылок (BFS): для каждой из m вершин ($m \leq S + 1$) перебираются все 26 букв, на каждую букву выполняется $O(1)$ операций $O(26m) = O(S)$.
- Построение выходных ссылок: обход по всем m вершинам $O(m) = O(S)$.
- Вычисление максимальных длин цепочек: рекурсивные функции с мемоизацией посещают каждую вершину не более двух раз (один раз для fail, один для out) $O(m) = O(S)$.

Все этапы линейны, доминирует множитель 26 из-за обхода всех букв в каждой вершине, но это константа.

Оценка по памяти

Память составляет $O(m * |A| + m) = O(S * |A|) = O(S)$ (при фиксированном алфавите).

- Бор (vector<Node> trie): каждая из m вершин хранит массив *next*[26] (26 целых чисел), *fail*, *out*, *term*. На одну вершину — $O(|A|)$ памяти всего $O(m * |A|) = O(S * 26) = O(S)$.
- Вспомогательные векторы: *memo_fail* и *memo_out* — по m целых чисел каждый $O(m) = O(S)$.
- Очередь BFS: в худшем случае содержит все m вершин $O(m) = O(S)$.
- Исходные образцы: суммарно занимают $O(S)$.

Таким образом, память линейно зависит от суммарной длины образцов.

Тестирование

Задание 1

Таблица 1 – Тестирование задания 1

№ теста	Входные данные	Выходные данные
Тест 1: Нет ни одного паттерна (n=0). Проверка, что программа не выводит ничего, даже если текст не пуст.	ACGT 0	(пустой вывод)
Тест 2: Паттерн длиннее текста. Проверка, что вхождение не находится.	ACGT 1 ACGTAC	(пустой вывод)
Тест 3: Паттерн содержит символ 'N' (входит в алфавит). Проверка корректного сопоставления с символом 'N' в тексте.	ACNGT 1 CN	2 1
Тест 4: Два паттерна, один является суффиксом другого. Проверка выходных ссылок.	AGAG 2 AG GAG	1 1 2 2 3 1
Тест 5: Текст содержит символ, не входящий в алфавит (например, 'X'), который преобразуется в 'A'. Проверка, что это может привести к ложному срабатыванию	AXA 1 AA	Error: 'X' symbol not in alfabet

Задание 2

Таблица 2 – Тестирование задания 2

№ теста	Входные данные	Выходные данные
Тест 1: Шаблон состоит только из джокеров. Любая подстрока длины m совпадает.	ACGT ?? ?	1 2 3
Тест 2. Шаблон без джокеров (обычный поиск подстроки).	ACGTACGT ACG ?	1 5
Тест 3. Шаблон с джокерами в начале, середине и конце. Проверка правильного учёта сдвигов сегментов.	ACGTACGT ?CG? ?	1 5

Тест 4. Длина шаблона больше длины текста – совпадений быть не может.	AC ACG? ?	(пустой вывод)
Тест 5. Шаблон содержит символ N (входит в алфавит), но в тексте нет ни N, ни G – проверка, что N не интерпретируется как джокер.	AAA N?G ?	(пустой вывод)

Задание 3

Таблица 3 – Тестирование задания 3

№ теста	Входные данные	Выходные данные
Тест 1: Пустой набор образцов (n=0). Бор состоит только из корня. Цепочки суффиксных и выходных ссылок имеют нулевую длину.	0 0	0 0
Тест 2. Один образец из одного символа "a". Есть две вершины: корень и терминальная. Суффиксная ссылка из терминала ведёт в корень (длина 1), выходных ссылок нет.	0 1 a	1 0
Тест 3. Один длинный образец "aaa...a" (100 символов). Вершины выстроены в цепочку. Суффиксные ссылки идут последовательно, максимальная глубина равна длине образца. Выходных ссылок нет, так как только последняя вершина терминальна, но её fail-ссылка указывает на нетерминальную.	0 1 aaaaaaaaaaaaaaaaaaaaa aaaaaaaaaaaaaaaaaaaaa aaaaaaaaaaaaaaaaaaaaa aaaaaaaaaaaaaaaaaaaaa aaaaaaaaaaaaa	100 0
Тест 4. Три образца: "a", "aa", "aaa". Бор образует цепочку из трёх терминальных вершин. Суффиксные ссылки: 1-0, 2-1, 3-2 - max_fail = 3. Выходные ссылки: 3-2, 2-1, 1 нет - max_out = 2.	0 3 a aa aaa	3 2
Тест 5. Образцы с перекрёстными	0 3 ab b abab	3 2

суффиксными связями: "ab", "b", "abab". Проверяет, что максимальная длина цепочки выходных ссылок может быть меньше, чем по суффиксным ссылкам.		
---	--	--

Программы для всех случаев отработала корректно.

Вывод

В ходе выполнения лабораторной работы были реализованы три программы, решающие задачи множественного точного поиска подстрок, поиска с джокером на основе разбиения шаблона на сегменты и вычисления максимальных длин цепочек суффиксных и выходных ссылок в автомате Ахо-Корасик. Для каждой задачи разработаны соответствующие структуры данных, проведён анализ временной и ёмкостной сложности (линейная зависимость от суммарной длины образцов и текста), а также выполнено тестирование на граничных случаях, подтвердившее корректность работы алгоритмов. Полученные результаты полностью удовлетворяют требованиям задания.

Приложение

Код программы:

1.cpp

```
#include <iostream>
#include <vector>
#include <queue>
#include <string>
#include <algorithm>
#include <cstring>
#include <fstream>

using namespace std;

struct Node {
    int next[5];
    int fail;
    int out;
    vector<int> pat;
    Node() {
        memset(next, -1, sizeof(next));
        fail = 0;
        out = -1;
    }
};

int char_to_idx(char c) {
    switch (c) {
        case 'A': return 0;
        case 'C': return 1;
        case 'G': return 2;
        case 'T': return 3;
        case 'N': return 4;
        default: return -1;
    }
}

char idx_to_char(int i) {
    switch (i) {
        case 0: return 'A';
        case 1: return 'C';
        case 2: return 'G';
        case 3: return 'T';
        case 4: return 'N';
        default: return '?';
    }
}

void print_node(ofstream& out, const vector<Node>& nodes, int v, const
string& label = "") {
    if (!label.empty()) out << label << " ";
    out << "Node " << v << ":\n";
    out << "  next: ";
    for (int c = 0; c < 5; ++c) {
        out << idx_to_char(c) << "->" << nodes[v].next[c];
        if (c < 4) out << ", ";
    }
    out << "\n  fail = " << nodes[v].fail;
    out << "\n  out  = " << nodes[v].out;
    out << "\n  pat  = [ ";
```

```

        for (int p : nodes[v].pat) out << p << " ";
        out << "]\n";
    }

    int main(int argc, char* argv[]) {
        ios::sync_with_stdio(false);
        cin.tie(nullptr);

        bool logfile = true;

        ofstream logfile_file;
        if (logfile) {
            logfile_file.open("log1.txt");
            if (!logfile_file.is_open()) {
                cerr << "Не удалось открыть logfile_output.txt для
записи\n";
                return 1;
            }
            logfile_file << "Отладочный вывод: \n\n";
        }

        string T;
        cin >> T;
        int n;
        cin >> n;
        vector<int> pattern_len(n + 1);
        vector<Node> nodes;
        nodes.reserve(225000);
        nodes.push_back(Node());

        if (logfile) {
            logfile_file << "Текст T: " << T << "\n";
            logfile_file << "Количество паттернов: " << n << "\n";
            logfile_file << "Паттерны:\n";
        }

        for (int idx = 1; idx <= n; ++idx) {
            string p;
            cin >> p;
            pattern_len[idx] = (int)p.size();
            if (logfile) logfile_file << " " << idx << ": " << p << "\n";

            int v = 0;
            for (char ch : p) {
                int c = char_to_idx(ch);
                if (c == -1) {
                    cout << "Error: '" << ch << "' symbol not in alfabet"
<< endl;
                    return 0;
                }
                if (nodes[v].next[c] == -1) {
                    nodes.push_back(Node());
                    nodes[v].next[c] = (int)nodes.size() - 1;
                    if (logfile) logfile_file << "        Создан новый узел "
<< nodes.size() - 1
<< " по переходу " <<
idx_to_char(c) << " из узла " << v << "\n";
                }
                v = nodes[v].next[c];
            }
            nodes[v].pat.push_back(idx);
            if (logfile) logfile_file << "        Паттерн " << idx << " добавлен
в узел " << v << "\n";
        }
    }

```

```

if (logfile) {
    logfile_file << "\nБор после вставки \n";
    for (size_t v = 0; v < nodes.size(); ++v) {
        print_node(logfile_file, nodes, v);
    }
}

queue<int> q;
for (int c = 0; c < 5; ++c) {
    if (nodes[0].next[c] != -1) {
        int v = nodes[0].next[c];
        nodes[v].fail = 0;
        nodes[v].out = -1;
        q.push(v);
        if (logfile) logfile_file << "Инициализация: узел " << v
            << " (переход по " << idx_to_char(c)
<< ") fail=0, out=-1\n";
    } else {
        nodes[0].next[c] = 0;
    }
}

if (logfile) logfile_file << "\n Построение fail-ссылок \n";

while (!q.empty()) {
    int u = q.front();
    q.pop();
    if (logfile) logfile_file << "Обрабатываем узел " << u << "\n";

    for (int c = 0; c < 5; ++c) {
        if (nodes[u].next[c] != -1) {
            int v = nodes[u].next[c];
            nodes[v].fail = nodes[nodes[u].fail].next[c];
            if (!nodes[nodes[v].fail].pat.empty())
                nodes[v].out = nodes[v].fail;
            else
                nodes[v].out = nodes[nodes[v].fail].out;

            if (logfile) {
                logfile_file << "    Переход по " << idx_to_char(c)
<< " -> узел " << v
                << ": fail = " << nodes[v].fail
                << ", out = " << nodes[v].out << "\n";
            }
            q.push(v);
        } else {
            nodes[u].next[c] = nodes[nodes[u].fail].next[c];
            if (logfile && nodes[u].next[c] != 0) {
                logfile_file << "    Переназначен переход " <<
idx_to_char(c)
                << " узла " << u << " -> " <<
nodes[u].next[c] << "\n";
            }
        }
    }
}

if (logfile) {
    logfile_file << "\n Финальное состояние автомата \n";
    for (size_t v = 0; v < nodes.size(); ++v) {
        print_node(logfile_file, nodes, v);
    }
}

```

```

vector<vector<int>>> ans(T.size() + 1);
int state = 0;

if (logfile) logfile_file << "\n обход текста \n";

for (int i = 0; i < (int)T.size(); ++i) {
    char ch = T[i];
    int c = char_to_idx(ch);
    if (c == -1) {
        cout << "Error: '" << ch << "' symbol not in alfabet" <<
endl;
        return 0;
    }
    int old_state = state;
    state = nodes[state].next[c];

    if (logfile) {
        logfile_file << "Позиция " << i+1 << " (" << ch << "): "
        << "состояние " << old_state << " -> " << state
<< "\n";
    }

    int pos = i + 1;

    if (!nodes[state].pat.empty()) {
        if (logfile) logfile_file << "   Найдены паттерны в узле "
<< state << ": ";
        for (int idx : nodes[state].pat) {
            int start = pos - pattern_len[idx] + 1;
            ans[start].push_back(idx);
            if (logfile) logfile_file << idx << " (старт " << start
<< ") ";
        }
        if (logfile && !nodes[state].pat.empty()) logfile_file <<
"\n";
    }

    int out = nodes[state].out;
    int depth = 0;
    while (out != -1) {
        if (!nodes[out].pat.empty()) {
            if (logfile) logfile_file << "   out[" << depth << "] =
" << out << " -> паттерны: ";
            for (int idx : nodes[out].pat) {
                int start = pos - pattern_len[idx] + 1;
                ans[start].push_back(idx);
                if (logfile) logfile_file << idx << " (старт " <<
start << ") ";
            }
            if (logfile && !nodes[out].pat.empty()) logfile_file
<< "\n";
        }
        out = nodes[out].out;
        ++depth;
    }

    if (logfile) {
        logfile_file << "\nРезультаты поиска\n";
    }

    for (int i = 1; i <= (int)T.size(); ++i) {
        if (!ans[i].empty()) {

```

```

        sort(ans[i].begin(), ans[i].end());
        for (int idx : ans[i]) {
            cout << i << " " << idx << '\n';
            if (logfile) logfile_file << i << " " << idx << '\n';
        }
    }

    if (logfile) {
        logfile_file << "\nКонец\n";
        logfile_file.close();
    }

    return 0;
}

```

2.cpp

```

#include <iostream>
#include <vector>
#include <queue>
#include <string>
#include <cstring>
#include <fstream>

using namespace std;

const int ALPHABET_SIZE = 5;

int char_to_idx(char c) {
    switch (c) {
        case 'A': return 0;
        case 'C': return 1;
        case 'G': return 2;
        case 'T': return 3;
        case 'N': return 4;
        default: return -1;
    }
}

char idx_to_char(int i) {
    switch (i) {
        case 0: return 'A';
        case 1: return 'C';
        case 2: return 'G';
        case 3: return 'T';
        case 4: return 'N';
        default: return '?';
    }
}

struct Node {
    Node* son[ALPHABET_SIZE];
    Node* go[ALPHABET_SIZE];
    Node* parent;
    Node* suffLink;
    Node* up;
    char charToParent;
    bool isLeaf;
    vector<int> leafPatternNumber;

    Node() {
        memset(son, 0, sizeof(son));
        memset(go, 0, sizeof(go));
    }
}

```

```

        parent = nullptr;
        suffLink = nullptr;
        up = nullptr;
        isLeaf = false;
    }
};

Node* root;

Node* getLink(Node* v, char c);
Node* getUp(Node* v);

Node* getSuffLink(Node* v) {
    if (v->suffLink == nullptr) {
        if (v == root || v->parent == root) {
            v->suffLink = root;
        } else {
            v->suffLink = getLink(getSuffLink(v->parent),
v->charToParent);
        }
    }
    return v->suffLink;
}

Node* getLink(Node* v, char c) {
    int idx = char_to_idx(c);
    if (idx == -1) {
        cout << "Error: '" << c << "' symbol not in alfabet" << endl;
        return 0;
    }
    if (v->go[idx] == nullptr) {
        if (v->son[idx]) {
            v->go[idx] = v->son[idx];
        } else if (v == root) {
            v->go[idx] = root;
        } else {
            v->go[idx] = getLink(getSuffLink(v), c);
        }
    }
    return v->go[idx];
}

Node* getUp(Node* v) {
    if (v->up == nullptr) {
        Node* suff = getSuffLink(v);
        if (suff->isLeaf) {
            v->up = suff;
        } else if (suff == root) {
            v->up = root;
        } else {
            v->up = getUp(suff);
        }
    }
    return v->up;
}

void addString(const string& s, int patternNumber) {
    Node* cur = root;
    for (size_t i = 0; i < s.length(); ++i) {
        char c = s[i];

```

```

        int idx = char_to_idx(c);
        if (idx == -1) {
            cout << "Error: '" << c << "' symbol not in alfabet" <<
endl;

            exit(0);
        }
        if (cur->son[idx] == nullptr) {
            cur->son[idx] = new Node();
            cur->son[idx]->parent = cur;
            cur->son[idx]->charToParent = c;
        }
        cur = cur->son[idx];
    }
    cur->isLeaf = true;
    cur->leafPatternNumber.push_back(patternNumber);
}

int main(int argc, char* argv[]) {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    bool logflag = false;
    for (int i = 1; i < argc; ++i) {
        if (strcmp(argv[i], "--logflag") == 0) {
            logflag = true;
            break;
        }
    }

    ofstream logfile;
    if (logflag) {
        logfile.open("log2.txt");
        if (!logfile) {
            cerr << "error: cannot open log2.txt for writing" << endl;
            return 1;
        }
        logfile << "Отладочный вывод алгоритма Ахо-Корасик (ленивая
рекурсия)" << endl;
    }

    string T, P, wc_str;
    cin >> T >> P >> wc_str;
    char joker = wc_str[0];

    if (logfile.is_open()) {
        logfile << "текст T = " << T << endl;
        logfile << "шаблон P = " << P << endl;
        logfile << "joker = " << joker << endl;
    }

    vector<int> pat_len;
    vector<int> pat_offset;
    int m = (int)P.size();

    for (int i = 0; i < m; ++i) {
        if (P[i] != joker) {
            int j = i;
            while (j < m && P[j] != joker) ++j;
            pat_offset.push_back(i);
            pat_len.push_back(j - i);
            i = j - 1;
        }
    }
}

```

```

int k = (int)pat_len.size();
if (logfile.is_open()) {
    logfile << "найдено сегментов: " << k << endl;
    for (int i = 0; i < k; ++i) {
        logfile << " сегмент " << i << ": offset=" << pat_offset[i]
            << ", длина=" << pat_len[i] << ", текст=\""
            << P.substr(pat_offset[i], pat_len[i]) << "\"\" <<
endl;
    }
}

root = new Node();
root->parent = root;
root->suffLink = root;
root->up = root;

for (int idx = 0; idx < k; ++idx) {
    string segment = P.substr(pat_offset[idx], pat_len[idx]);
    addString(segment, idx);
    if (logfile.is_open()) {
        logfile << "сегмент " << idx << " добавлен в бор" << endl;
    }
}

vector<int> cnt(T.size() + 5, 0);
int n = (int)T.size();

Node* cur = root;
for (int j = 0; j < n; ++j) {
    char c = T[j];
    int idx = char_to_idx(c);
    if (idx == -1) {
        cout << "Error: '" << c << "' symbol not in alfabet" <<
endl;
        return 0;
    }
    Node* prev = cur;
    cur = getLink(cur, c);

    if (logfile.is_open()) {
        logfile << " позиция " << j + 1 << " символ '" << c
            << "' -> переход " << prev << " -> " << cur << endl;
    }

    Node* tmp = cur;
    while (tmp != root) {
        if (tmp->isLeaf) {
            for (int segmentIdx : tmp->leafPatternNumber) {
                int start_frag = j - pat_len[segmentIdx] + 1;
                int start_candidate = start_frag -
pat_offset[segmentIdx];
                if (start_candidate >= 0 && start_candidate + m <=
n) {
                    cnt[start_candidate]++;
                    if (logfile.is_open()) {
                        logfile << " найден сегмент " <<
segmentIdx
                        << " (длина " <<
pat_len[segmentIdx]
                        << ", offset " <<
pat_offset[segmentIdx]
                        << ") -> позиция в тексте " <<
start_candidate + 1

```

```

<< " (cnt=" << cnt[start_candidate]
<< ")" << endl;
    }
    }
    }
    tmp = getUp(tmp);
}
}

if (logfile.is_open()) {
    logfile << "итоговые совпадения (начальные позиции в 1-
индексации):" << endl;
}

for (int i = 0; i + m <= n; ++i) {
    if (cnt[i] == k) {
        cout << i + 1 << '\n';
        if (logfile.is_open()) {
            logfile << " " << i + 1 << endl;
        }
    }
}

if (logfile.is_open()) {
    logfile << "отладка завершена." << endl;
    logfile.close();
}

return 0;
}

```

3.cpp

```

#include <iostream>
#include <vector>
#include <string>
#include <queue>
#include <algorithm>
#include <fstream>

using namespace std;

struct Node {
    int next[26];
    int fail;
    int out;
    bool term;
    Node() {
        fill(next, next + 26, -1);
        fail = -1;
        out = -1;
        term = false;
    }
};

int get_fail_len(int v, const vector<Node>& trie, vector<int>& memo) {
    if (v == 0) return 0;
    if (memo[v] != -1) return memo[v];
    memo[v] = 1 + get_fail_len(trie[v].fail, trie, memo);
    return memo[v];
}

int get_out_len(int v, const vector<Node>& trie, vector<int>& memo) {

```

```

        if (v == 0) return 0;
        if (trie[v].out == -1) return 0;
        if (memo[v] != -1) return memo[v];
        memo[v] = 1 + get_out_len(trie[v].out, trie, memo);
        return memo[v];
    }

    int main() {
        int log_flag;
        cin >> log_flag;
        ofstream log_file;
        if (log_flag) {
            log_file.open("log3.txt");
        }

        int n;
        cin >> n;
        vector<string> patterns(n);
        for (int i = 0; i < n; ++i) {
            cin >> patterns[i];
        }

        if (log_flag) {
            log_file << "начало построения бора\n";
            log_file << "количество образцов: " << n << "\n";
        }

        vector<Node> trie(1);

        for (int i = 0; i < n; ++i) {
            const string& p = patterns[i];
            if (log_flag) {
                log_file << "добавление образца " << p << "\n";
            }
            int v = 0;
            for (char ch : p) {
                int c = ch - 'a';
                if (trie[v].next[c] == -1) {
                    trie[v].next[c] = trie.size();
                    trie.emplace_back();
                    if (log_flag) {
                        log_file << "        создана новая вершина " <<
trie.size() - 1 << " из " << v << " по букве " << ch << "\n";
                    }
                }
                v = trie[v].next[c];
            }
            trie[v].term = true;
            if (log_flag) {
                log_file << "        вершина " << v << " помечена как
терминальная\n";
            }
        }

        if (log_flag) {
            log_file << "\нбор после добавления всех образцов:\n";
            for (size_t v = 0; v < trie.size(); ++v) {
                log_file << "вершина " << v << ": переходы ";
                bool first = true;
                for (int c = 0; c < 26; ++c) {
                    if (trie[v].next[c] != -1) {
                        if (!first) log_file << ", ";
                        log_file << char('a' + c) << " -> " <<
trie[v].next[c];
                    }
                }
            }
        }
    }

```

```

        first = false;
    }
}
if (trie[v].term) log_file << " (терминальная)";
log_file << "\n";
}
}

queue<int> q;
for (int c = 0; c < 26; ++c) {
    if (trie[0].next[c] != -1) {
        int v = trie[0].next[c];
        trie[v].fail = 0;
        q.push(v);
        if (log_flag) {
            log_file << "суффиксная ссылка вершины " << v << " ->
0 (корень)\n";
        }
    } else {
        trie[0].next[c] = 0;
    }
}

if (log_flag) {
    log_file << "\nпостроение суффиксных ссылок (обход в
ширину):\n";
}

while (!q.empty()) {
    int v = q.front();
    q.pop();
    for (int c = 0; c < 26; ++c) {
        int u = trie[v].next[c];
        if (u != -1) {
            trie[u].fail = trie[trie[v].fail].next[c];
            q.push(u);
            if (log_flag) {
                log_file << "суффиксная ссылка вершины " << u << "
-> " << trie[u].fail << "\n";
            }
        } else {
            trie[v].next[c] = trie[trie[v].fail].next[c];
        }
    }
}

if (log_flag) {
    log_file << "\nпостроение выходных ссылок:\n";
}
for (int v = 0; v < (int)trie.size(); ++v) {
    if (v == 0) {
        trie[v].out = -1;
        continue;
    }
    int f = trie[v].fail;
    if (trie[f].term) {
        trie[v].out = f;
    } else {
        trie[v].out = trie[f].out;
    }
    if (log_flag) {
        if (trie[v].out != -1) {
            log_file << "выходная ссылка вершины " << v << " -> "
<< trie[v].out << "\n";

```

```

        } else {
            log_file << "выходная ссылка вершины " << v << "
отсутствует\n";
        }
    }

    vector<int> memo_fail(trie.size(), -1);
    vector<int> memo_out(trie.size(), -1);
    int max_fail = 0;
    int max_out = 0;

    for (int v = 0; v < (int)trie.size(); ++v) {
        int flen = get_fail_len(v, trie, memo_fail);
        int olen = get_out_len(v, trie, memo_out);
        if (flen > max_fail) max_fail = flen;
        if (olen > max_out) max_out = olen;
    }

    if (log_flag) {
        log_file << "\nвычисление длин цепочек:\n";
        for (int v = 0; v < (int)trie.size(); ++v) {
            log_file << "вершина " << v << ": глубина по суффиксным
ссылкам = " << memo_fail[v];
            log_file << ", глубина по выходным ссылкам = " << memo_out[v]
<< "\n";
        }
        log_file << "\nрезультат:\n";
        log_file << "длина самой длинной цепочки из суффиксных ссылок
= " << max_fail << "\n";
        log_file << "длина самой длинной цепочки из конечных ссылок =
" << max_out << "\n";
    }

    cout << max_fail << " " << max_out << endl;

    if (log_flag) {
        log_file.close();
    }

    return 0;
}

```